Lecture 2: Introduction into Neural Networks (2)

Yaxin Bi
School of Computing
University of Ulster

1

1

Contents

- Inside neural nodes
- Activation functions
- Neural network architectures
- Error calculation
- Weight propagation in forward and backward
- Summary

2

2

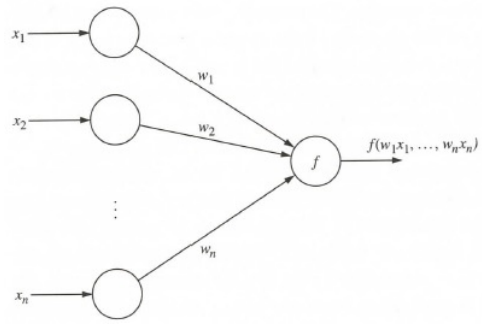
NN (simplified) from a graphical viewpoint

- Neural Network (NN) is a directed graph $F=<V, A>$ with vertices $V=\{1, 2, \ldots, n\}$ and arcs $A = \{<i, j> \mid 1 <= i, j <= n\}$, with the following restrictions:
- V is partitioned into a set of input nodes V_I, hidden nodes V_H, and output nodes V_O.
- The vertices are also partitioned into layers
- Any arc $<i, j>$ must have node i in layer h-1 and node j in layer h.
- Arc $<i, j>$ is labeled with a numeric value w_{ij}.
- Node i is labeled with a function f_i.

3

3

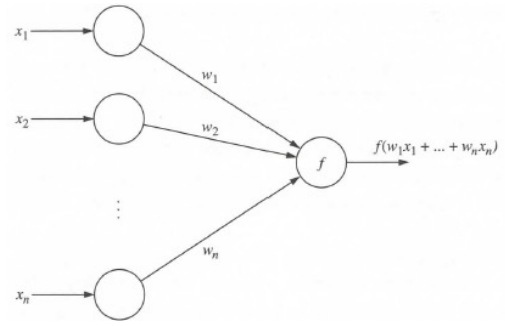
General neuron model



4

4

Weighted input summation

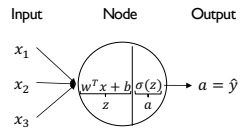


5

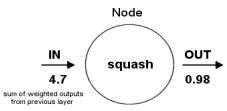
5

Single NN Node

- Activity rule in 2 steps
  - neuron summation (linear combiner)
  - Activation/transfer/squashing function



Squashing function limits the amplitude of node output, i.e., prevent the output value from being too large.

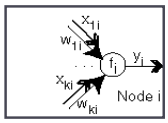


6

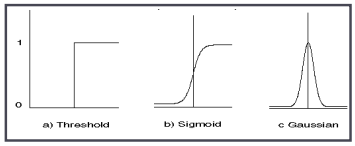
6

NN Activation/Transfer/Squashing Functions

- Functions associated with nodes in graph.
- Output may be in range [-1,1] or [0,1]



$$y_i = f_i(\sum_{j=1}^{k} w_{ji}\, x_{ji}) = f_i([w_{1i} \ldots w_{ki}] \begin{bmatrix} x_{1i} \\ \ldots \\ x_{ki} \end{bmatrix})$$



7

7

More Activation Functions (cont'd)

Linear:

$$f_i(S) = c\, S$$

Threshold or Step:

$$f_i(S) = \left\{ \begin{array}{ll} 1 & if S > T \\ 0 & otherwise \end{array} \right\}$$

Ramp:

$$f_i(S) = \left\{ \begin{array}{ll} 1 & if S > T_2 \\ \frac{S-T_1}{T_2-T_1} & if T_1 \leq S \leq T_2 \\ 0 & if S < T_1 \end{array} \right\}$$

Sigmoid:

$$f_i(S) = \frac{1}{(1+e^{-c\,S})}$$

Hyperbolic Tangent:

$$f_i(S) = \frac{(1-e^{-S})}{(1+e^{-c\,S})}$$

Gaussian:

$$f_i(S) = e^{\frac{-S^2}{v}}$$

8

8

More Activation Functions (cont'd)

Sigmoid
$\sigma(x) = \frac{1}{1+e^{-x}}$

Leaky ReLU
$\max(0.1x, x)$

tanh
$\tanh(x)$

Maxout
$\max(w_1^T x + b_1, w_2^T x + b_2)$

ReLU
$\max(0, x)$

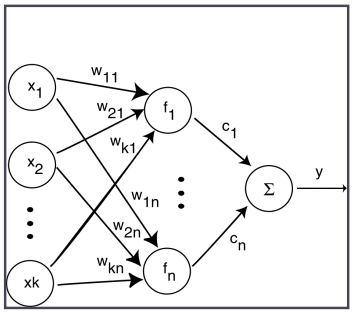
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

9

9

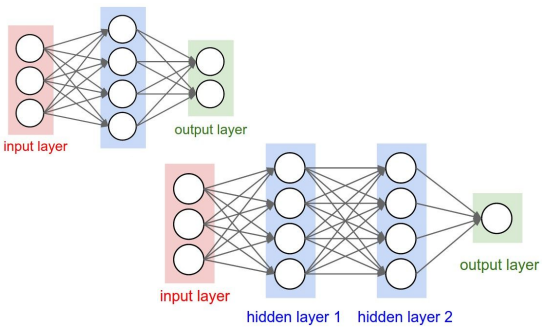
Radial Basis Function Network



10

10

Neural networks: Architectures



11

11

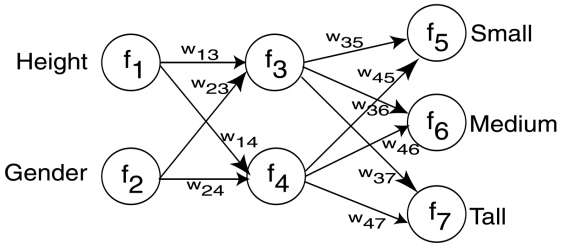
Height Example Data

Name	Gender	Height	Output1	Output2
Kristina	F	1.6m	Short	Medium
Jim	M	2m	Tall	Medium
Maggie	F	1.9m	Medium	Tall
Martha	F	1.88m	Medium	Tall
Stephanie	F	1.7m	Short	Medium
Bob	M	1.85m	Medium	Medium
Kathy	F	1.6m	Short	Medium
Dave	M	1.7m	Short	Medium
Worth	M	2.2m	Tall	Tall
Steven	M	2.1m	Tall	Tall
Debbie	F	1.8m	Medium	Medium
Todd	M	1.95m	Medium	Medium
Kim	F	1.9m	Medium	Tall
Amy	F	1.8m	Medium	Medium
Wynette	F	1.75m	Medium	Medium

12

12

Neural Network Example

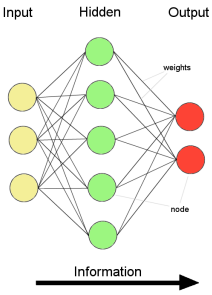


13

13

A general net structure (feedforward)

- A Neural Network generally maps a set of inputs to a set of outputs via a hidden layer
  - Information flow is unidirectional
- Number of inputs/outputs is variable
- The Network itself is composed of an arbitrary number of nodes with an arbitrary topology
- Information is distributed
- Information processing is parallel



14

14

NN Learning

- Propagate input values through graph.
- Compare output to desired output.
- Adjust weights in graph accordingly.

15

15

Building a NN Model

- A NN Model is a computational model consisting of three parts:
  - Neural Network graph
  - Learning algorithm that indicates how learning takes place.
  - Recall techniques that determine how information is obtained from the network.
- We will look at propagation as the recall technique.

16

16

Building a NN Issues

- Number of source nodes
- Number of hidden layers
- Training data
- Number of sinks
- Interconnections
- Weights
- Activation Functions
- Learning Technique
- When to stop learning

17

17

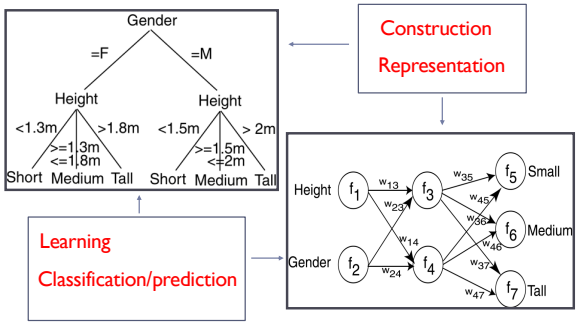
Classification Using Neural Networks

- Typical NN structure for classification:
  - One output node per class
  - Output value is class membership function value
- Supervised learning
- For each tuple in training set, propagate it through NN. Adjust weights on edges to improve future classification.
- Algorithms: Propagation, Backpropagation, Gradient Descent

18

18

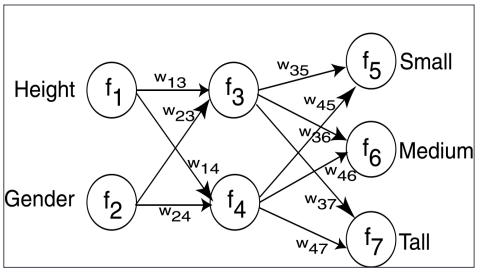
Decision Tree vs. Neural Network



19

19

Propagation



20

20

NN Propagation Algorithm

Input:
N //Neural Network
$X = < x_1, ..., x_h >$ //Input tuple consisting of values for input attributes only
Output:
$Y = < y_1, ..., y_m >$ //Tuple consisting of output values from NN
Propagation Algorithm:
//Algorithm illustrates propagation of a tuple through a NN
for each node i in the input layer do
Output x_i on each output arc from i;
for each hidden layer do
for each node i do
$S_i = (\sum_{j=1}^{k} (w_{ji}\, x_{ji}))$;
for each output arc from i do
Output $\frac{(1-e^{-cS_i})}{(1+e^{-cS_i})}$;
for each node i in the output layer do
$S_i = (\sum_{j=1}^{k} (w_{ji}\, x_{ji}))$;
Output $y_i = \frac{1}{(1+e^{-cS_i})}$;

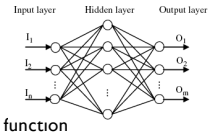
21

21

Typical three-layered neural network structure

- Each neuron in the hidden layer receives an activation signal, which is the weighted sum of all the inputs entering the neuron

$$S_i = \sum_{j=1}^{k} w_{ji}\, x_{ji}$$



- Produces an output through a transfer function

$$y_i = \frac{(1 + e^{-S_i})}{(1 + e^{-cS_i})}$$

- The neurons in the output layer receive the following activation signals from the hidden neurons

$$y_i = \frac{1}{(1 + e^{-cS_i})}$$

- W_{jk} is the weight of the connection between the neurons j and k in the hidden and output layers, respectively.

22

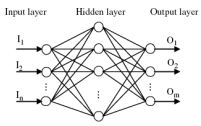
22

Typical three-layered neural network structure (cont'd)

- These activation signals are transformed again to give the outputs of the neural network

$$o_k = y_k = \frac{1}{(1 + e^{-cS_k})}$$

- which are predicted values of the outputs, to be compared with their desired or actual values.
- The error function at the output neurons is defined as



$$E(W) = \frac{1}{2} \sum_{k} (d_k - o_k)^2,$$

- where d_k and o_k are the desired and predicted values of the outputs, respectively. The error function should be minimized so that the neural network achieves the best performance.

23

23

NN Learning

- Adjust weights to perform better with the associated test data.
- Supervised: Use feedback from knowledge of correct classification.
- Unsupervised: No knowledge of correct classification needed.

24

24

NN Supervised Learning

Input:
  N //Starting Neural Network
  X //Input tuple from Training Set
  D //Output tuple desired
Output:
  N //Improved Neural Network
SupLearn Algorithm:
  //Simplistic algorithm to illustrate approach to NN learning
  Propagate X through N producing output Y;
  Calculate error by comparing D to Y;
  Update weights on arcs in N to reduce error;

25

25

Supervised Learning

- Possible error values assuming output from node *i* is y_i but should be d_i:

$$| y_i - d_i |$$

$$\frac{(y_i - d_i)^2}{2}$$

$$\sum_{i=1}^{m} \frac{(y_i - d_i)^2}{m}$$

- Change weights on arcs based on estimated error

26

26

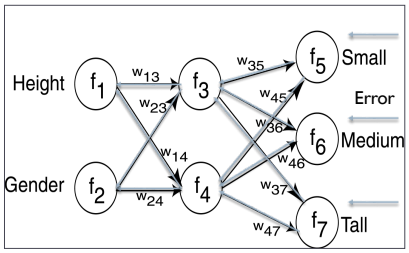
NN Backpropagation

- Propagate changes to weights backward from output layer to input layer.
- Delta Rule: $\triangle w_{ij} = c\, x_{ij} (d_j - y_j)$
- Gradient Descent: technique to modify the weights in the graph.

27

27

Backpropagation



28

28

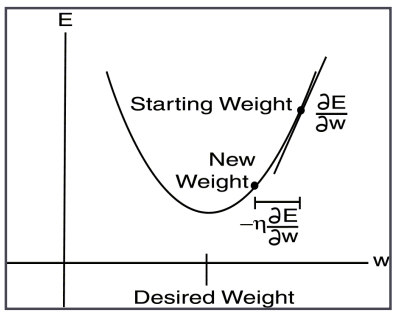
Backpropagation Algorithm

Input:
  N //Starting Neural Network
  $X =< x_1, ..., x_h >$ //Input tuple from Training Set
  $D =< d_1, ..., d_m >$ //Output tuple desired
Output:
  N //Improved Neural Network
Backpropagation Algorithm:
  //Illustrate backpropagation
  Propagation(N, X);
  $E = 1/2 \sum_{i=1}^{m} (d_i - y_i)^2$;
  Gradient(N,E);

29

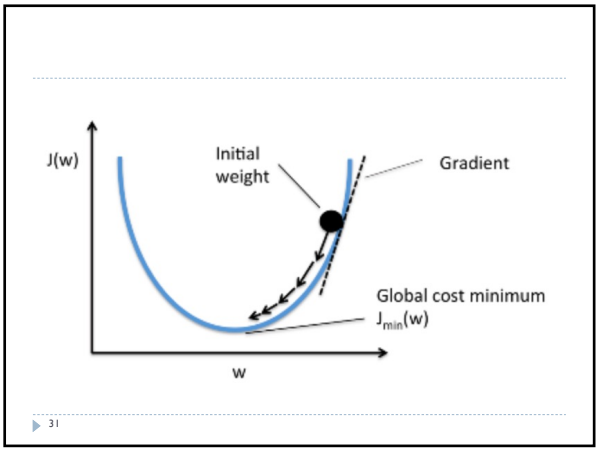
29

Gradient Descent



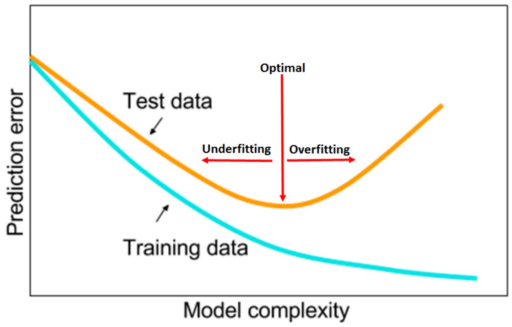
30

30



31

Optimal capacity, falling between underfitting and overfitting



32

Gradient Descent Algorithm

Input:
N //Starting Neural Network
E //Error found from Back algorithm
Output:
N //Improved Neural Network
Gradient Algorithm:
//Illustrates incremental gradient descent
for each node i **in output layer do**
for each node j **input to** i **do**
$\Delta w_{ji} = \eta\ (d_i - y_i)\ y_j\ (1 - y_i)\ y_i$;
$w_{ji} = w_{ji} + \Delta w_{ji}$;
layer = previous layer;
for each node j **in this layer do**
for each node k **input to** j **do**
$\Delta w_{kj} = \eta\ y_k\ \frac{1-(y_j)^2}{2} \sum_m (d_m - y_m)\ w_{jm}\ y_m(1 - y_m)$;
$w_{kj} = w_{kj} + \Delta w_{kj}$;

33

33

Output Layer Learning

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = -\eta \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial S_i} \frac{\partial S_i}{\partial w_{ji}}$$

$$\frac{\partial y_i}{\partial S_i} = \frac{\partial}{\partial S_i}\left(\frac{1}{(1+e^{-S_i})}\right) = (1 - (\frac{1}{1+e^{-S_i}}))\ (\frac{1}{1+e^{-S_i}}) = (1 - y_i)y_i$$

$$\frac{\partial S_i}{\partial w_{ji}} = y_j.$$

$$\frac{\partial E}{\partial y_i} = \frac{\partial}{\partial y_i}(1/2 \sum_m (d_m - y_m)^2) = -(d_i - y_i)$$

$$\Delta w_{ji} = \eta\ (d_i - y_i)\ y_j\ (1 - \frac{1}{1+e^{-S_i}})\ \frac{1}{1+e^{-S_i}} = \eta\ (d_i - y_i)\ y_j\ (1 - y_i)\ y_i$$

34

34

Hidden Layer Learning

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}}$$

$$\frac{\partial E}{\partial w_{kj}} = \sum_m \frac{\partial E}{\partial y_m} \frac{\partial y_m}{\partial S_m} \frac{\partial S_m}{\partial y_j} \frac{\partial y_j}{\partial S_j} \frac{\partial S_j}{\partial w_{kj}}.$$

$$\frac{\partial E}{\partial y_m} = -(d_m - y_m)$$

$$\frac{\partial y_m}{\partial S_m} = (1 - y_m)y_m$$

$$\frac{\partial S_m}{\partial y_j} = w_{jm}$$

$$\frac{\partial y_j}{\partial S_j} = \frac{\partial}{\partial S_j}(\frac{(1 - e^{-S_j})}{(1 + e^{-S_j})}) = \frac{(1 + (\frac{1-e^{-S_j}}{1+e^{-S_j}}))\ (1 - (\frac{1-e^{-S_j}}{1+e^{-S_j}}))}{2} = \frac{1 - y_j^2}{2}$$

$$\frac{\partial S_j}{\partial w_{kj}} = y_k$$

$$\Delta w_{kj} = \eta\ y_k\ \frac{1 - (y_j)^2}{2} \sum_m (d_m - y_m)\ w_{jm}\ y_m\ (1 - y_m)$$

35

35

Types of NNs

- Different NN structures used for different problems.
- Perceptron
- Radial Basis Function Network
- Self Organizing Feature Map (omitted)

36

36

Radial Basis Function Network

- RBF function has Gaussian shape
- RBF Networks
  - Three Layers
  - Hidden layer – Gaussian activation function
  - Output layer – Linear activation function

37

37

Generating Rules from NNs

```
Algorithm 0.1
Input:
   D      //Training data
   N      //Initial Neural Network
Output:
   R      //Derived rules
RX Algorithm:
          //Rule Extraction algorithm to extract rules from NN
   cluster output node activation values;
   cluster hidden node activation values;
   generate rules that describe the output values in terms of the hidden activation values;
   generate rules that describe hidden output values in terms of inputs;
   Combine the two sets of rules.
```

38

38

NN Advantages

- Learning
- Can continue learning even after training set has been applied.
- Easy parallelization
- Solves many problems

39

39

NN Disadvantages

- Difficult to understand
  - The model tends to be a black box or input/output table without analytical basis.
  - Its connection weights are not easy to be explained.
- The sample size has to be large.
- May suffer from over-fitting
- Structure of graph must be determined a priori.
- Input values must be numeric.
- Verification difficult.

40

40

Dimensions of a Neural Network

- Various types of neurons
- Various network architectures
- Various learning algorithms
- Various applications

41

41